



uscribe User Guide

Contents

1	Introduction	3
1.1	What uscribe is for	3
1.2	A taste of Unicon highlighting	3
1.3	Relation to unidoc	4
1.4	How to read this guide	4
2	Installation	5
2.1	Requirements	5
2.2	TeX / PDF dependencies by platform	5
2.2.1	macOS (Homebrew)	5
2.2.2	Debian / Ubuntu	6
2.2.3	Fedora / RHEL-family	6
2.2.4	Arch Linux	6
2.2.5	Windows	6
2.2.6	What uscribe LaTeX output needs	7
2.3	Build uscribe	7
2.4	Verify	7
2.5	Build this manual	7
3	Quick Start	9
3.1	1. Create a manifest	9
3.2	2. Write chapters	9
3.3	3. Book settings	10
3.4	4. Build HTML	10
3.5	5. Build PDF (optional)	11
3.6	Where files go	11
4	Markup Reference	12
4.1	Headings	12

4.2	Document fields	12
4.3	Paragraphs and lists	13
4.4	Definition lists	13
4.5	Tables	14
4.6	Labels and cross-references	16
4.7	Stable labels	16
4.8	Inline markup	16
4.9	Directives	17
4.10	Include / literalinclude	19
4.11	Unicon listings	20
5	Themes	21
5.1	Built-in themes	21
5.2	Build-time default	21
5.3	Theme layout on disk	22
6	Command Line	23
6.1	Synopsis	23
6.2	Options	23
6.3	Outputs	24
6.4	Exit status	25

Chapter 1

Introduction

uscribe is a prose/book documentation generator for Unicon. It reads a small RST-like markup, builds a doctree, resolves cross-chapter references, and writes themed HTML (and optionally LaTeX/PDF) for Unicon documentation and books.

This manual is the user guide for *uscribe* itself. Sources live under `doc/uscribe/` in the unicon tree and are built *with* *uscribe*.

1.1 What *uscribe* is for

- Multi-chapter books and manuals (ordered by a manifest file)
- HTML output with sidebar navigation, search, and selectable themes
- Unicon code samples with syntax highlighting
- Images and simple admonitions

It is **not** a full RST/docutils toolchain. The markup subset is intentionally small; extend the parser as real documents need more.

1.2 A taste of Unicon highlighting

Mark listings with `.. code-block:: unicon` (also `icon` or `icn`). In HTML they are highlighted client-side:

```
procedure greet(who)
  write("Hello, ",           who | "world", "!")
end
```

```
procedure main()
  greet("uscribe")
  greet()
end
```

See [Markup Reference](#) for the full dialect and [Quick Start](#) to generate a tiny book.

1.3 Relation to unidoc

`uni/unidoc/` extracts API docs from Unicon source comments. *uscribe* is a separate tool for authored prose. They can coexist; a future `.. api::` bridge may connect them.

1.4 How to read this guide

- [Installation](#) — build and install the `uscribe` binary
- [Quick Start](#) — generate HTML from a tiny book
- [Markup Reference](#) — headings, lists, inline markup, directives
- [Themes](#) — basic, classic, dark, and the in-page switcher
- [Command Line](#) — CLI options

Chapter 2

Installation

2.1 Requirements

Required

- A built Unicon tree (`./configure && make` from the unicon sources)

Optional (PDF only)

- A TeX engine on your PATH: `pdflatex`, `xelatex`, or `lualatex`. `uscribe` picks the first one it finds. HTML and `-format=latex` (`.tex` only) do not need TeX.

There is no Homebrew or apt package literally named `pdflatex`; that name is a program shipped inside a TeX distribution. Install one of the packages below, then confirm:

```
which pdflatex || which xelatex || which lualatex
```

2.2 TeX / PDF dependencies by platform

2.2.1 macOS (Homebrew)

Compact (recommended for `uscribe`):

```
brew install --cask basictex
```

Full MacTeX (much larger):

```
brew install --cask mactex
# or: brew install --cask mactex-no-gui
```

After BasicTeX or MacTeX, restart the terminal (or run `eval "$(/usr/libexec/path_helper)"`) so the new binaries are on `PATH`.

2.2.2 Debian / Ubuntu

Minimal (often enough):

```
sudo apt update
sudo apt install texlive-latex-base
```

Recommended if `pdflatex` complains about missing packages (`hyperref`, `fancyvrb`, `graphicx` fonts, and similar):

```
sudo apt install texlive-latex-recommended texlive-fonts-recommended
```

Larger kitchen-sink options:

```
sudo apt install texlive-latex-extra
# or everything: sudo apt install texlive-full
```

2.2.3 Fedora / RHEL-family

```
sudo dnf install texlive-latex
# fuller: sudo dnf install texlive-scheme-medium
# everything: sudo dnf install texlive-scheme-full
```

2.2.4 Arch Linux

```
sudo pacman -S texlive-basic texlive-latex
# or a broader set: sudo pacman -S texlive-meta
```

2.2.5 Windows

Install either MiKTeX (<https://miktex.org/download>) — packages on demand — or TeX Live for Windows (<https://tug.org/texlive/>).

Ensure the installer puts `pdflatex` on your `PATH`, then open a new Command Prompt or PowerShell window.

2.2.6 What uscribe LaTeX output needs

Generated `STEM.tex` uses a small set of packages (`graphicx`, `hyperref`, `xcolor`, `fancyvrb`, `booktabs`, `longtable`). `multirow` is loaded when the `.sty` is installed (needed for grid-table row spans). A latex recommended install is usually enough; GitHub Pages installs `texlive-latex-extra` so the user-guide PDF build has the full set. If `pdflatex` stops with `File ...sty not found`, install the matching TeX Live or MiKTeX package (or a larger scheme) and retry.

SVG figures are not embedded in PDF; convert them to PDF or PNG, or accept the boxed placeholder uscribe emits for `.svg` paths.

2.3 Build uscribe

From the unicon tree:

```
cd uni/uscribe
make
```

That compiles the `uscribe` package, links `./uscribe`, and installs a copy to `../../bin/uscribe`.

Regenerate Make dependencies after adding `.icn` files:

```
make deps
```

2.4 Verify

```
uscribe
# prints usage and exits
```

2.5 Build this manual

HTML:

```
cd doc/uscribe
make
# open out/index.html
```

PDF (after installing a TeX engine as above):


```
cd doc/uscribe
make pdf
# open out/uscribe-userguide.pdf
```

`make latex` writes `out/uscribe-userguide.tex` without running a TeX engine. `make pdf` writes the `.tex` and runs the engine twice (TOC and cross-references).

Chapter 3

Quick Start

A uscribe project is a directory of chapter `.rst` files, a manifest that lists them in reading order, and usually a `book.conf` for book-wide title, logo, and PDF name.

3.1 1. Create a manifest

`book.manifest` (name is conventional; any path works):

```
# comments and blank lines are ignored
preface.rst
chapter1.rst
chapter2.rst
```

3.2 2. Write chapters

Underline-style headings, paragraphs, lists, and directives. Example `chapter1.rst`:

```
First Chapter
=====

Hello from uscribe. See :ref:'Second Chapter' later.

Second Section
-----

- alpha
- beta
```

```
.. note::

    Indented body of a note.

.. code-block:: unicon

    procedure main()
        write("hi")
    end
```

Underline rule: the underline must be at least as long as the title. Shorter underlines are not treated as headings.

Here is that Unicon listing as it appears when built (syntax highlighted in HTML):

```
procedure main()
    write("hi")
end
```

3.3 3. Book settings

`book.conf` next to the manifest (optional; CLI overrides):

```
title: My Book
copyright: 2026, Your Name
logo: images/logo.png
name: my-book
theme: basic
themePath: /path/to/uni/uscribe/themes
```

3.4 4. Build HTML

```
uscribe --targetDir=./out
```

Or from a project Makefile patterned on `doc/uscribe/Makefile`.

Open `out/index.html`. Use the sidebar for chapters, search, and theme switching.

3.5 5. Build PDF (optional)

Install a TeX engine first — see the *TeX / PDF dependencies by platform* section under [Installation](#). Then:

```
uscribe --format=pdf --targetDir=./out
```

Or make `pdf`. Output is `out/STEM.pdf` from the `name` key (default stem `book`). This user guide uses `name: uscribe-userguide`. SVG figures are not embedded; convert them to PDF or PNG for TeX.

3.6 Where files go

- `*.rst` — chapter sources
- `*.manifest` — chapter order
- `book.conf` — book title, logo, copyright, PDF name, theme
- `images/` — figures (copied into `out/images/`)
- `out/` — generated HTML and/or `STEM.tex` / `STEM.pdf`
- `themes/` — built-in skins shared across projects (HTML)

Chapter 4

Markup Reference

uscribe accepts a **restricted RST-like** dialect. Unknown constructs are either ignored, treated as plain text, or left as raw directives.

4.1 Headings

```
Chapter Title
=====

Section
-----

Subsection
~~~~~
```

Characters = - ~ ^ are recognized as underline styles. Heading level follows the underline character. The underline length must be greater than or equal to the title length.

4.2 Document fields

A chapter may start with an RST field list (Docutils bibliographic fields). Repeat `:author:` for several names. Book chapters show these as a byline under the first heading; omit the fields and nothing is added. Reports also use `:title:`, `:trnumber:`, `:abstract:`, and `:docclass:` for the UTR cover.

```
:author: Jane Doe
:author: Pat Lee
```

```
:date: 2026-08-15
:copyright: 2026, Jane Doe
```

```
Chapter Title
=====
```

Book-wide title, logo, theme, and PDF name belong in `book.conf` next to the manifest (CLI options override). They are not chapter fields.

4.3 Paragraphs and lists

Blank lines separate paragraphs. Bullet lists use `-` or `*` with a following space. Numbered lists use `1.` / `1)` or auto-number `#.` . Indented continuation lines belong to the current item.

Source:

```
1. Install the ‘‘uscribe’’ binary (:doc:‘install’)
2. Write a tiny book (:doc:‘quickstart’)
#. Pick a theme (:doc:‘themes’)
```

Rendered:

1. Install the `uscribe` binary ([Installation](#))
2. Write a tiny book ([Quick Start](#))
3. Pick a theme ([Themes](#))

4.4 Definition lists

A term on its own line followed by an indented body. Blank lines inside the body start a new paragraph.

Source:

```
format
  Selects the builder.

  Use ‘‘html’’ for a browsable book, or ‘‘latex’’ / ‘‘pdf’’ for print.

theme
  Names the HTML skin (‘‘basic’’, ‘‘classic’’, or ‘‘dark’’).
```

Rendered:

format Selects the builder.

Use `html` for a browsable book, or `latex` / `pdf` for print.

theme Names the HTML skin (`basic`, `classic`, or `dark`).

4.5 Tables

Simple tables use `=` column separators. The first row is the header. PDF output uses `longtable` so tables can break across pages.

Source:

```
=====
Flag    Meaning
=====
html    HTML book
latex   ‘‘book.tex’’
pdf     run a TeX engine
=====
```

Rendered:

Flag	Meaning
html	HTML book
latex	“book.tex
pdf	run a TeX

Grid tables use `+-+` borders; a border with `=` marks the header. Omit an internal `|` to span columns:

Source:

```
+-----+-----+-----+
| Left  | Mid   | Right |
+=====+=====+=====+
| A     | B spanning two |
+-----+-----+-----+
| C spanning two | D     |
+-----+-----+-----+
```

Rendered:

Left	Mid	Right
A	B spanning two	
C spanning two		D

A plain grid without spans:

Source:

```
+-----+-----+
| Flag   | Meaning   |
+=====+=====+
| html   | HTML book  |
+-----+-----+
| latex  | book.tex   |
+-----+-----+
```

Rendered:

Flag	Meaning
html	HTML book
latex	book.tex

List tables use the `list-table` directive:

Source:

```
.. list-table::
   :header-rows: 1

   * - Flag
     - Meaning
   * - html
     - HTML book
   * - pdf
     - run a TeX engine
```

Rendered:

Flag	Meaning
html	HTML book
pdf	run a TeX engine

4.6 Labels and cross-references

Place `.. _name:` immediately before a heading, figure, or table. Pass 1 collects explicit labels and title slugs; pass 2 turns ref roles into `chapter.html#anchor` links.

Source:

```
.. _stable-labels:

Stable labels
-----

A preceding ‘‘.. _name:’’ attaches to the next heading. Link with a
ref role naming that label, optionally with display text in angle
brackets.
```

Rendered:

4.7 Stable labels

A preceding `.. _name:` attaches to the next heading. Link with a ref role naming that label, optionally with display text in angle brackets.

Live refs (explicit label and display-text form):

- plain: [Stable labels](#)
- with display text: [Jump here](#)
- chapter: [Installation](#)

Title-only refs (matching a heading exactly) still work. Full chapter sources are linked from each HTML footer as **source** (and **txt** in the report index Formats column).

4.8 Inline markup

- single asterisks for *emphasis*
- double asterisks for **strong**
- backticks for `inline literals`

4.9 Directives

Admonition bodies are re-parsed as nested blocks (lists, code, nested directives). Source then rendered:

```
.. note::

    Admonition bodies can hold nested blocks:

    - a bullet list
    - and even a nested tip

    .. tip::

        Nested directives are parsed, not flattened.
```

Note.

Admonition bodies can hold nested blocks:

- a bullet list
- and even a nested tip

Tip.

Nested directives are parsed, not flattened.

Also: `warning`, `tip`, `important`, `caution`, `attention`, `danger`, `error`, `hint`.

Figures take a caption (numbered in HTML/LaTeX) and optional `:name:` / preceding `.. _label::`

```
.. _logo-figure:

.. figure:: images/uscribe-logo.png

    :alt: uscribe logo

    The uscribe mark.
```

See The uscribe mark..

Other authoring forms (shown literally):



Figure 4.1: The uscribe mark.

```

.. code-block:: unicon

    procedure main()
        write("ok")
    end

.. image:: images/diagram.svg

    Optional alt text

.. include:: path/to/fragment.rst

.. literalinclude:: path/to/file.icn

    :language: unicon
    :lines: 1-10
    :start-after: marker
    :end-before: marker
    :dedent: 3

```

Paths are resolved next to the including chapter, then under the process cwd. Nested `include` is allowed; circular includes warn and are skipped.

4.10 Include / literalinclude

`literalinclude` shows a file as a listing; `include` parses it as markup. The tip below is the same fragment both ways.

Source (`includes/shared-tip.rst`):

```

.. tip::

    Snippets under ‘‘includes/’’ can be pulled into any chapter with
    ‘‘.. include::’’ so the same advice is not copy-pasted.

```

Rendered:

Tip.

Snippets under `includes/` can be pulled into any chapter with
`.. include::` so the same advice is not copy-pasted.

Pull a real `.icn` file as a listing. Language defaults from the extension (`.icn` $\rightarrow$ `unicon`); override with `:language::`

```
# Shared sample for literalinclude demos
procedure main()
  write("hello from includes/hello.icn")
  write(&version)
end
```

The same file, only the body between markers:

```
write("hello from includes/hello.icn")
write(&version)
```

4.11 Unicon listings

code-block languages `unicon`, `icon`, and `icn` get Unicon syntax highlighting in HTML (via `highlight-unicon.js`). Other languages are emitted as plain `<pre><code class="language-...">` without Unicon highlighting — use `sh` for shell, `rst` or `text` for markup samples. `literalinclude` of `.icn` files uses the same highlighting.

Example Unicon program as it appears in the built book:

```
# Hello from a uscribe listing
procedure main()
  every i := 1 to 3 do
    write("tick ", i)
  write(&version)
end
```

A line ending in `::` (RST literal-block introducer) followed by an indented block is also treated as a code listing. `Unicon::` becomes the label `Unicon:` plus the indented tree; a lone `::` introduces a block with no label.

Unknown `.. name::` directives become HTML comments so content is not silently dropped.

See also [Quick Start](#) and [Command Line](#).

Chapter 5

Themes

HTML chrome (sidebar, search, prev/next) is fixed in the generator. A *theme* is a CSS skin plus shared JavaScript.

5.1 Built-in themes

- **basic** — light sidebar (default)
- **classic** — serif / warm-gray documentation look
- **dark** — dark background

All three are installed into `out/_static/` as `theme-NAME.css`. Readers pick a theme from the **Theme** dropdown in the sidebar; the choice is stored in the browser (`localStorage` key `uscribe-theme`).

5.2 Build-time default

```
uscribe ... --theme=classic --themePath=../themes
```

Or with the sample Makefiles:

```
make          # THEME=basic
make classic
make dark
```

`-theme` only sets the *initial* stylesheet until the reader selects another in the UI.

5.3 Theme layout on disk

```
themes/  
  basic/static/book.css  
  classic/static/book.css  
  dark/static/book.css  
  _shared/search.js  
  _shared/highlight.css  
  _shared/highlight-unicorn.js  
  _shared/theme-switcher.js
```

To add a custom theme, create `themes/mytheme/static/book.css` and pass `-theme=mytheme`. Keep the same HTML class names (`.sidebar`, `.main`, `.tok-keyword`, ...) so shared scripts keep working.

See `themes/README.md` in the source tree for layout details.

Chapter 6

Command Line

6.1 Synopsis

```
uscribe [options]
```

6.2 Options

****--manifest=FILE**** Plain-text chapter list: one source path per line. Lines starting with `#` and blank lines are ignored. Paths are relative to the process current working directory. Default: `book.manifest` in the current directory.

****--config=FILE**** Book-level settings file. If omitted, uscribe loads `book.conf` next to the manifest when that file exists. Keys: `title`, `copyright`, `logo`, `name`, `theme`, `themePath`, `docclass`. Paths in the file are relative to the config file. Any command-line option overrides the file.

****--title=TITLE**** Book title for `<title>`, the sidebar heading, the index page, and the LaTeX `\\title`. Default: from `book.conf`, else Untitled Book.

****--copyright=TEXT**** Optional footer copyright (text after ©), e.g. 2026, Jafar Al-Gharaibeh. HTML only. Every HTML page always ends with **Powered by uscribe**; chapter pages also link **pdf** (the book PDF from `-name`, or a sibling PDF for reports) and **source** (`_sources/name.rst.txt`, viewable as plain text in the browser).

- **--name=STEM**** Basename for the book `.tex` / `.pdf` (default: `book`). HTML footers and the index link to `STEM.pdf`. A trailing `.pdf` or `.tex` is stripped. Letters, digits, `_`, and `-` only. Ignored for report collections (one file per report, named like the HTML stem).
- **--logo=FILE**** Optional logo. HTML copies it into `targetDir/_static/` and shows it in the sidebar. LaTeX / PDF copies it into `targetDir` and places it on the title page (page 1), not as a chapter figure.
- **--format=FMT**** Output format: `html` (default), `latex`, or `pdf`.
- `html` — one page per chapter plus index/search/themes. Report manifests get a UTR index table on `index.html`. - `latex` — for books, a single `STEM.tex` (default `book.tex`); for report manifests (every chapter is a report), one `.tex` per report - `pdf` — like `latex`, then run `pdflatex`, `xelatex`, or `lualatex` (whichever is first in `PATH`) twice for TOC/xrefs
- **--targetDir=DIR**** Output directory. Default: `./htmlBook` for HTML, `./latexBook` for latex/pdf. Created if missing.
- **--theme=NAME**** Initial HTML theme: `basic`, `classic`, or `dark` (or a custom name under `-themePath`). Default: `basic`. Ignored for latex/pdf.
- **--themePath=DIR**** Directory that contains theme subdirectories (`DIR/basic/static`, ...). If omitted, uscribe searches `./themes`, `../themes`, and `themes`. HTML only.

6.3 Outputs

HTML (`-format=html`): for each chapter `path/name.rst`, writes `targetDir/name.html`, plus:

- `index.html` — contents landing page (chapter list, or a UTR table with a **Formats** column of `pdf` / `txt` for report collections)
- `search.html` + `searchindex.js` — full-text search
- `_static/` — theme CSS, search JS, highlighter, theme switcher
- `_sources/` — chapter sources as `name.rst.txt` (browser-viewable text; linked as `txt` / **source**)

LaTeX / PDF: book manifests write `targetDir/STEM.tex` (and `STEM.pdf` with `-format=pdf`; default stem `book`). Report manifests write one `.tex` / `.pdf` per report, named like the HTML stem (`ex-report.tex` beside `ex-report.html`). Local `.. image::` files are copied into `targetDir` automatically (relative paths preserved). SVG is not embedded in LaTeX (a boxed placeholder is emitted instead — convert to PDF/PNG for TeX).

6.4 Exit status

Usage errors and missing files call `stop()` (non-zero). A successful build prints `wrote ...` lines for each output file.